

CSL599: LAB 3

GROUP CHAT APPLICATION

TEAM MEMBERS:

1. AGASTYA NATH 12340140
2. GALABA VAMSI 12340770
3. PARITOSH LAHRE 12341550
4. Y. RAHUL DEV REDDY 12342390

CLIENT URL: <http://10.1.75.51:7266/>

GITHUB REPO:

<https://github.com/SandstromPL/real-time-Group-Chat-Application.git>

TECH STACK:

- Python
- Websockets
- React
- Vite with ESLint

PROJECT STRUCTURE:

chat-app/

```
|— backend
|   |— server.py
|— chat-frontend
|   |— eslint.config.js
|   |— index.html
|   |— package.json
|   |— public
|   |   |— favicon.svg
|   |   |— icons.svg
|   |— README.md
|   |— src
|   |   |— App.css
|   |   |— App.jsx
|   |   |— App1.jsx
|   |   |— assets
|   |   |   |— hero.png
|   |   |   |— react.svg
|   |   |   |— vite.svg
```

```
|
|
|   |— index.css
|   |— main.jsx
|   |— vite.config.js
|   |
|   └─ README.md
```

FRONTEND

1. App.jsx

```
import { useEffect, useRef, useState } from "react";
import "./App.css";

const WS_URL = "ws://10.1.75.51:5265";

function App() {
  const [username, setUsername] = useState("");
  const [connected, setConnected] = useState(false);
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState("");

  const socketRef = useRef(null);

  function connect() {
    const name = username.trim();

    if (!name) {
      return;
    }

    const socket = new WebSocket(WS_URL);
    socketRef.current = socket;

    socket.onopen = () => {
      // Your backend expects the username as
      // the FIRST message after connection.
    }
  }
}
```

```
    socket.send(name);
};

socket.onmessage = (event) => {
    const message = event.data;

    // Backend sends validation errors like:
    // ERROR: Username already taken.
    if (message.startsWith("ERROR:")) {
        alert(message);
        socket.close();
        return;
    }

    setMessages((previous) => [...previous, message]);
    setConnected(true);
};

socket.onclose = () => {
    setConnected(false);
};

socket.onerror = () => {
    console.error("WebSocket error");
};
}

function sendMessage() {
    const message = input.trim();

    if (!message || !socketRef.current) {
        return;
    }

    if (socketRef.current.readyState === WebSocket.OPEN) {
```

```

        socketRef.current.send(message);
        setInput("");
    }
}

function handleKeyDown(event) {
    if (event.key === "Enter") {
        sendMessage();
    }
}

function disconnect() {
    socketRef.current?.close();
    socketRef.current = null;
    setMessages([]);
    setConnected(false);
}

if (!connected) {
    return (
        <div className="app">
            <div className="login-box">
                <h1>Group Chat</h1>

                <input
                    type="text"
                    placeholder="Enter username"
                    value={username}
                    onChange={(event) =>
setUsername(event.target.value)}
                    onKeyDown={(event) => {
                        if (event.key === "Enter") {
                            connect();
                        }
                    }}
                />
            </div>
        </div>
    );
}

```

```

        maxLength={20}
      />

      <button onClick={connect}>
        Join Chat
      </button>
    </div>
  </div>
);
}

return (
  <div className="app">
    <div className="chat-container">

      <header>
        <h1>Group Chat</h1>
        <span>Logged in as <b>{username}</b></span>

        <button onClick={disconnect}>
          Leave
        </button>
      </header>

      <div className="messages">
        {messages.map((message, index) => (
          <div className="message" key={index}>
            {message}
          </div>
        ))}
      </div>

      <div className="input-area">
        <input
          type="text"

```

```

        placeholder="Type a message..."
        value={input}
        onChange={(event) => setInput(event.target.value)}
        onKeyDown={handleKeyDown}
      />

      <button onClick={sendMessage}>
        Send
      </button>
    </div>

  </div>
</div>
);
}

export default App;

```

2. App.css

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f2f2f2;
}

.app {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

```

```
}

.login-box {
  width: 350px;
  padding: 30px;
  background: white;
  border-radius: 10px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
}

.login-box h1 {
  text-align: center;
  margin-bottom: 25px;
}

input {
  width: 100%;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 6px;
  font-size: 16px;
}

button {
  padding: 12px 18px;
  border: none;
  border-radius: 6px;
  background: #333;
  color: white;
  cursor: pointer;
  font-size: 15px;
}

.login-box button {
  width: 100%;
```

```
margin-top: 15px;
}

.chat-container {
width: 700px;
height: 600px;
background: white;
border-radius: 10px;
display: flex;
flex-direction: column;
overflow: hidden;
box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
}

header {
padding: 15px 20px;
background: #333;
color: white;
display: flex;
align-items: center;
gap: 15px;
}

header h1 {
margin: 0;
margin-right: auto;
}

header button {
background: #666;
}

.messages {
flex: 1;
overflow-y: auto;
```

```
padding: 20px;
}

.message {
padding: 10px 14px;
margin-bottom: 10px;
background: #eee;
border-radius: 6px;
}

.input-area {
display: flex;
gap: 10px;
padding: 15px;
border-top: 1px solid #ddd;
}

.input-area input {
flex: 1;
}

.input-area button {
width: 80px;
}
```

3. index.css

```
:root {
--text: #6b6375;
--text-h: #08060d;
--bg: #fff;
--border: #e5e4e7;
--code-bg: #f4f3ec;
--accent: #aa3bff;
--accent-bg: rgba(170, 59, 255, 0.1);
}
```

```
--accent-border: rgba(170, 59, 255, 0.5);
--social-bg: rgba(244, 243, 236, 0.5);
--shadow:
  rgba(0, 0, 0, 0.1) 0 10px 15px -3px, rgba(0, 0, 0, 0.05) 0
4px 6px -2px;

--sans: system-ui, 'Segoe UI', Roboto, sans-serif;
--heading: system-ui, 'Segoe UI', Roboto, sans-serif;
--mono: ui-monospace, Consolas, monospace;

font: 18px/145% var(--sans);
letter-spacing: 0.18px;
color-scheme: light dark;
color: var(--text);
background: var(--bg);
font-synthesis: none;
text-rendering: optimizeLegibility;
-webkit-font-smoothing: antialiased;
-moz-osx-font-smoothing: grayscale;

@media (max-width: 1024px) {
  font-size: 16px;
}
}

@media (prefers-color-scheme: dark) {
  :root {
    --text: #9ca3af;
    --text-h: #f3f4f6;
    --bg: #16171d;
    --border: #2e303a;
    --code-bg: #1f2028;
    --accent: #c084fc;
    --accent-bg: rgba(192, 132, 252, 0.15);
    --accent-border: rgba(192, 132, 252, 0.5);
```

```

    --social-bg: rgba(47, 48, 58, 0.5);
    --shadow:
        rgba(0, 0, 0, 0.4) 0 10px 15px -3px, rgba(0, 0, 0, 0.25)
0 4px 6px -2px;
}

#social .button-icon {
    filter: invert(1) brightness(2);
}
}

body {
    margin: 0;
}

#root {
    width: 1126px;
    max-width: 100%;
    margin: 0 auto;
    text-align: center;
    border-inline: 1px solid var(--border);
    min-height: 100svh;
    display: flex;
    flex-direction: column;
    box-sizing: border-box;
}

h1,
h2 {
    font-family: var(--heading);
    font-weight: 500;
    color: var(--text-h);
}

h1 {

```

```
font-size: 56px;
letter-spacing: -1.68px;
margin: 32px 0;
@media (max-width: 1024px) {
  font-size: 36px;
  margin: 20px 0;
}
}
h2 {
  font-size: 24px;
  line-height: 118%;
  letter-spacing: -0.24px;
  margin: 0 0 8px;
  @media (max-width: 1024px) {
    font-size: 20px;
  }
}
p {
  margin: 0;
}
code,
.counter {
  font-family: var(--mono);
  display: inline-flex;
  border-radius: 4px;
  color: var(--text-h);
}
code {
  font-size: 15px;
  line-height: 135%;
  padding: 4px 8px;
  background: var(--code-bg);
}
```

4. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

5. index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>chat-frontend</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

6. eslint.config.js

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
```

```

import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
  },
])

```

7. package.json

```

{
  "name": "chat-frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8"
  }
}

```

```
},  
  "devDependencies": {  
    "@eslint/js": "^10.0.1",  
    "@types/react": "^19.2.17",  
    "@types/react-dom": "^19.2.3",  
    "@vitejs/plugin-react": "^6.0.4",  
    "eslint": "^10.8.0",  
    "eslint-plugin-react-hooks": "^7.1.1",  
    "eslint-plugin-react-refresh": "^0.5.3",  
    "globals": "^17.7.0",  
    "vite": "^8.2.0"  
  }  
}
```

8. vite.config.js

```
import { defineConfig } from 'vite'  
import react from '@vitejs/plugin-react'  
  
// https://vite.dev/config/  
export default defineConfig({  
  plugins: [react()],  
})
```

BACKEND

1. server.py

```
import asyncio
import websockets
from websockets.exceptions import ConnectionClosed

HOST = "0.0.0.0"
PORT = 5000

# Maps each WebSocket connection to its username.
users = {}

async def broadcast(message):
    """Send a message to all currently connected clients."""
    if not users:
        return

    clients = list(users.keys())

    results = await asyncio.gather(
        *(client.send(message) for client in clients),
        return_exceptions=True,
    )

    # Remove clients whose connection failed while
    broadcasting.
    for client, result in zip(clients, results):
        if isinstance(result, Exception):
            users.pop(client, None)

    async def register_user(websocket):
```

```

"""Receive and validate a username from a new client."""
try:
    username = await websocket.recv()
except ConnectionClosed:
    return None

username = username.strip()

if not username:
    await websocket.send("ERROR: Username cannot be
empty.")
    return None

if len(username) > 20:
    await websocket.send("ERROR: Username must be 20
characters or fewer.")
    return None

# Usernames are considered unique case-insensitively.
existing_names = {
    name.lower()
    for name in users.values()
}

if username.lower() in existing_names:
    await websocket.send("ERROR: Username already taken.")
    return None

users[websocket] = username

return username

async def unregister_user(websocket):
    """Remove a client from the connected users."""

```

```
    return users.pop(websocket, None)

async def handle_client(websocket):
    """Handle the complete session of one connected client."""

    username = await register_user(websocket)

    if username is None:
        await websocket.close()
        return

    print(f"{username} joined the chat.")

    await broadcast(f"{username} joined the chat.")

    try:
        async for message in websocket:
            message = message.strip()

            if not message:
                continue

            print(f"{username}: {message}")

            await broadcast(f"{username}: {message}")

    except ConnectionClosed:
        pass

    finally:
        removed_username = await unregister_user(websocket)

        if removed_username is not None:
            print(f"{removed_username} left the chat.")
```

```
        await broadcast(f"{removed_username} left the  
chat.")
```

```
async def main():
```

```
    """Start the WebSocket server."""
```

```
    async with websockets.serve(  
        handle_client,  
        HOST,  
        PORT,  
    ):
```

```
        print(f"WebSocket server running on
```

```
ws://{HOST}:{PORT}")
```

```
        print("Waiting for clients...")
```

```
    ):
```

```
        print(f"WebSocket server running on
```

```
ws://{HOST}:{PORT}")
```

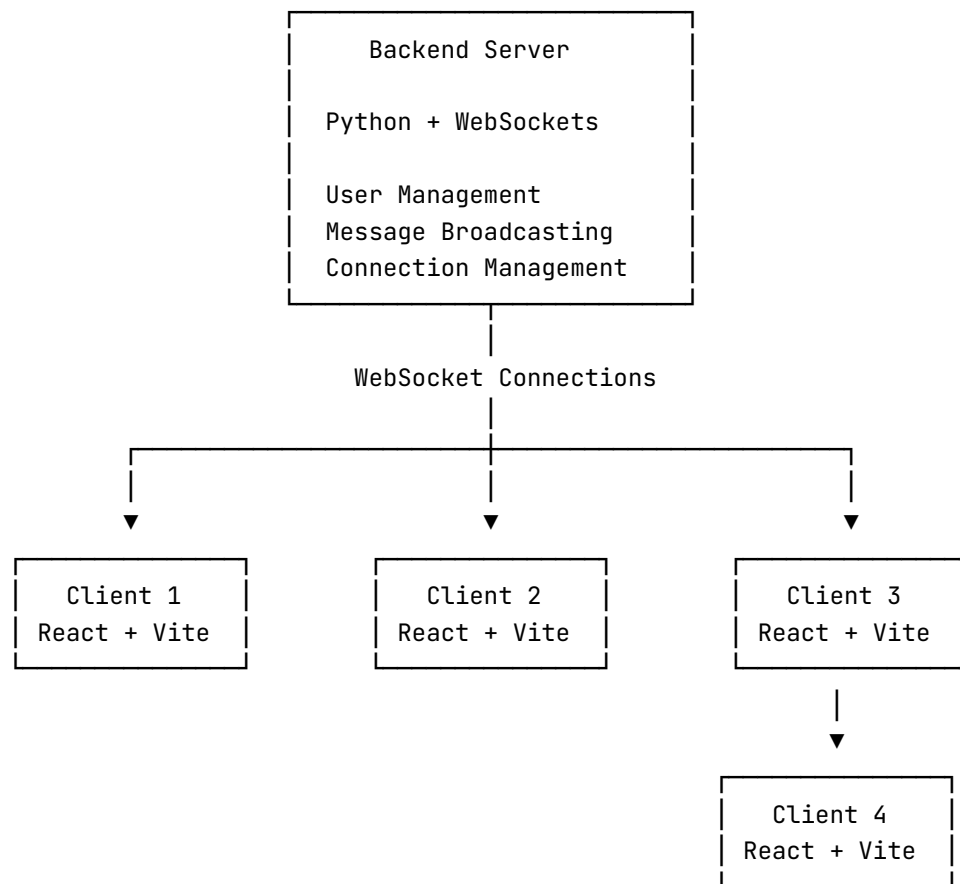
```
        print("Waiting for clients...")
```

```
    await asyncio.Future()
```

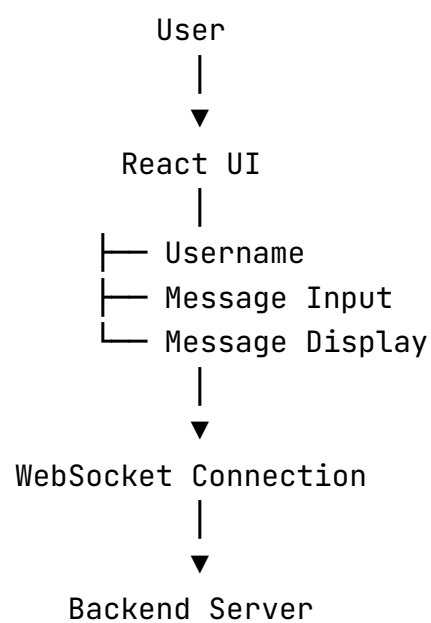
```
if __name__ == "__main__":
```

```
    asyncio.run(main())
```

SYSTEM ARCHITECTURE



FLOW:





10.1.75.51:7266



15



Group Chat

Logged in as
Vamsi

Leave

Vamsi joined the chat.

Vamsi: Hi

Vamsi: Who else is here

Vamsi: Anyone?

shey: meee

Vamsi: Yeaaaaa

Vamsi: Take screenshot of this and add in
report

Vamsi: The mobile compitable also

Type a message...

Send

